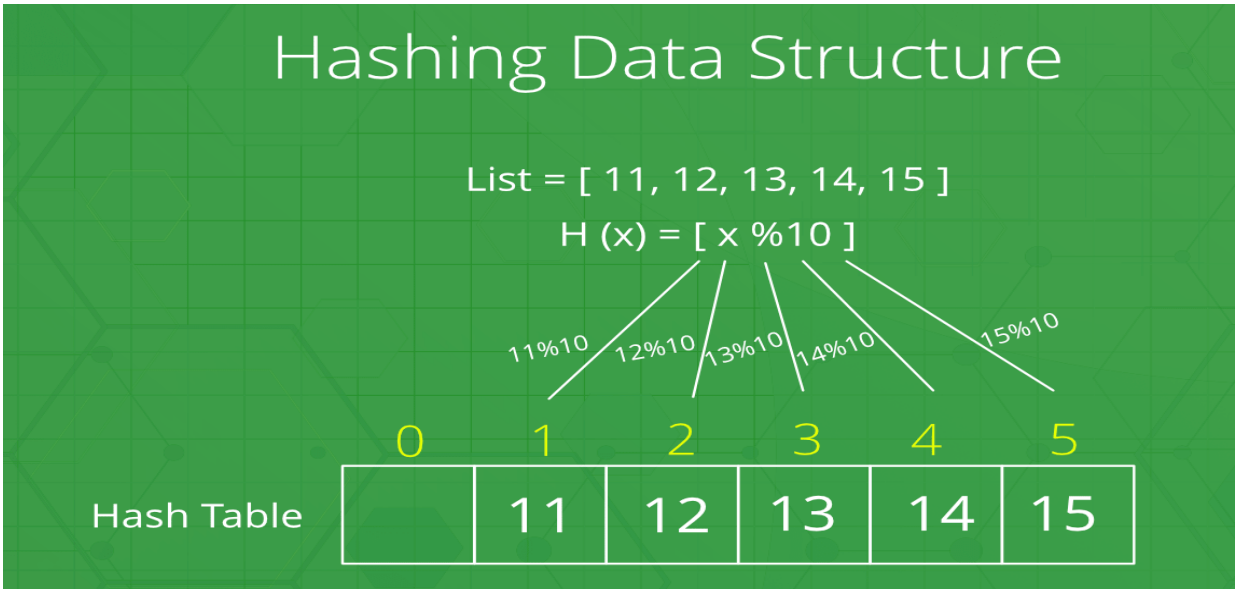
DBMS – Hashing

For a huge database structure, it can be almost next to impossible to search all the index values through all its level and then reach the destination data block to retrieve the desired data. **Hashing is an effective technique which is uses a hash function to find the exact location of a data record on the disc without using index structure in a minimum amount of time.**

Hashing uses hash functions with search keys as parameters to generate the address of a data record.



Why do we need Hashing?

In DBMS, hashing is a technique to directly search the location of desired data on the disk without using index structure. Data is stored in the form of data blocks whose address is generated by applying a hash function in the memory location where these records are stored known as a data block or data bucket.

Here, are the situations in the DBMS where you need to apply the Hashing method:

For a huge database structure, it's tough to search all the index values through all its level and then you need to reach the destination data block to get the desired data.

Hashing method is used to index and retrieve items in a database as it is faster to search that specific item using the shorter hashed key instead of using its original value.

Hashing is an ideal method to calculate the direct location of a data record on the disk without using index structure.

It is also a helpful technique for implementing dictionaries.

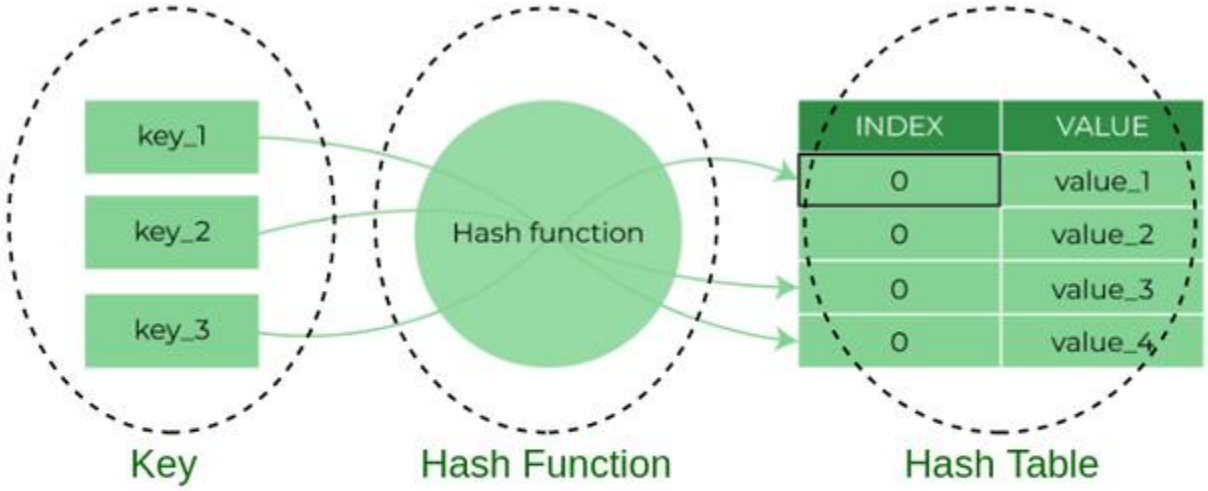
Why is Hashing Better than Indexing?

Hashing is better than Indexing because hashing helps calculate the location of the data that is present in the disk without the help of index structure.

Components of Hashing

There are majorly three components of hashing:

1. **Key:** A **Key** can be anything string or integer which is fed as input in the hash function the technique that determines an index or location for storage of an item in a data structure.
2. **Hash Function:** The **hash function** receives the input key and returns the index of an element in an array called a hash table. The index is known as the **hash index**.
3. **Hash Table:** Hash table is a data structure that maps keys to values using a special function called a hash function. Hash stores the data in an associative manner in an array where each data value has its own unique index.

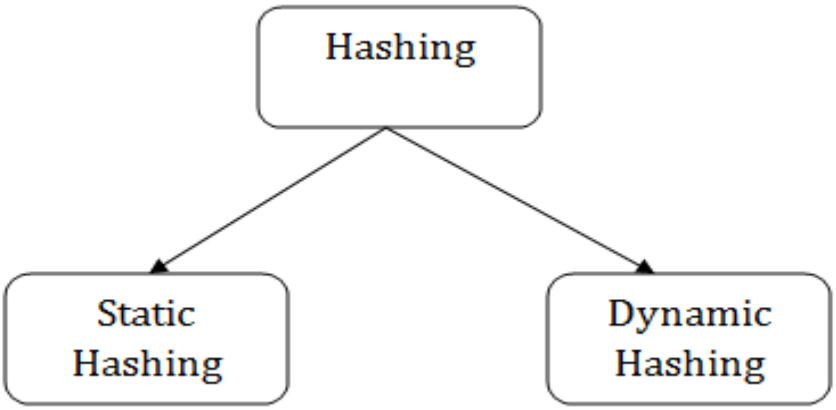


Components of Hashing

Hash Organization

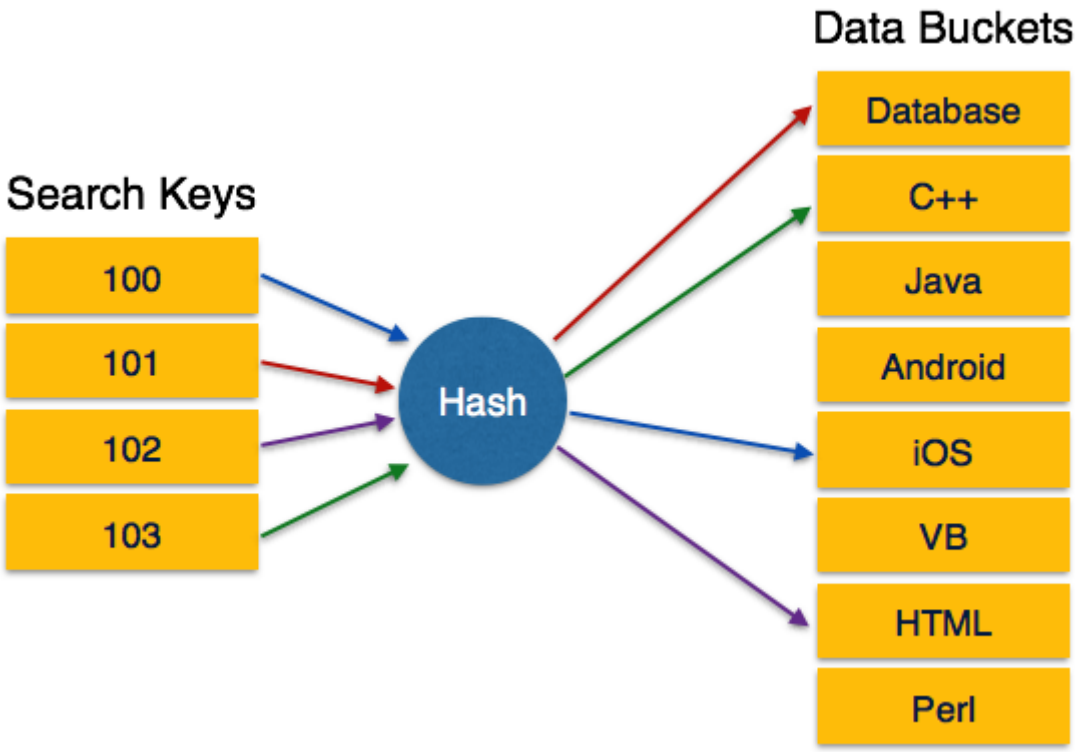
- **Bucket** − A hash file stores data in bucket format. Bucket is considered a unit of storage. A bucket typically stores one complete disk block, which in turn can store one or more records.
- **Hash Function** − A hash function, **h,** is a mapping function that maps all the set of search-keys **K** to the address where actual records are placed. It is a function from search keys to bucket addresses.

Types of Hashing:



Static Hashing

In static hashing, when a search-key value is provided, the hash function always computes the same address. For example, if we want to generate an address for STUDENT_ID = 104 using a mod (5) hash function, it always results in the same bucket address 4. There will not be any changes to the bucket address here. Hence a number of data buckets in the memory for this static hashing remain constant throughout.



Operation

- **Insertion** − When a record is required to be entered using static hash, the hash function **h** computes the bucket address for search key **K**, where the record will be stored.
  Bucket address = h(K)
- **Search** − When a record needs to be retrieved, the same hash function can be used to retrieve the address of the bucket where the data is stored.
- **Delete** − This is simply a search followed by a deletion operation.
- **Update** − To update a record, we will first search it using a hash function, and then the data record is updated

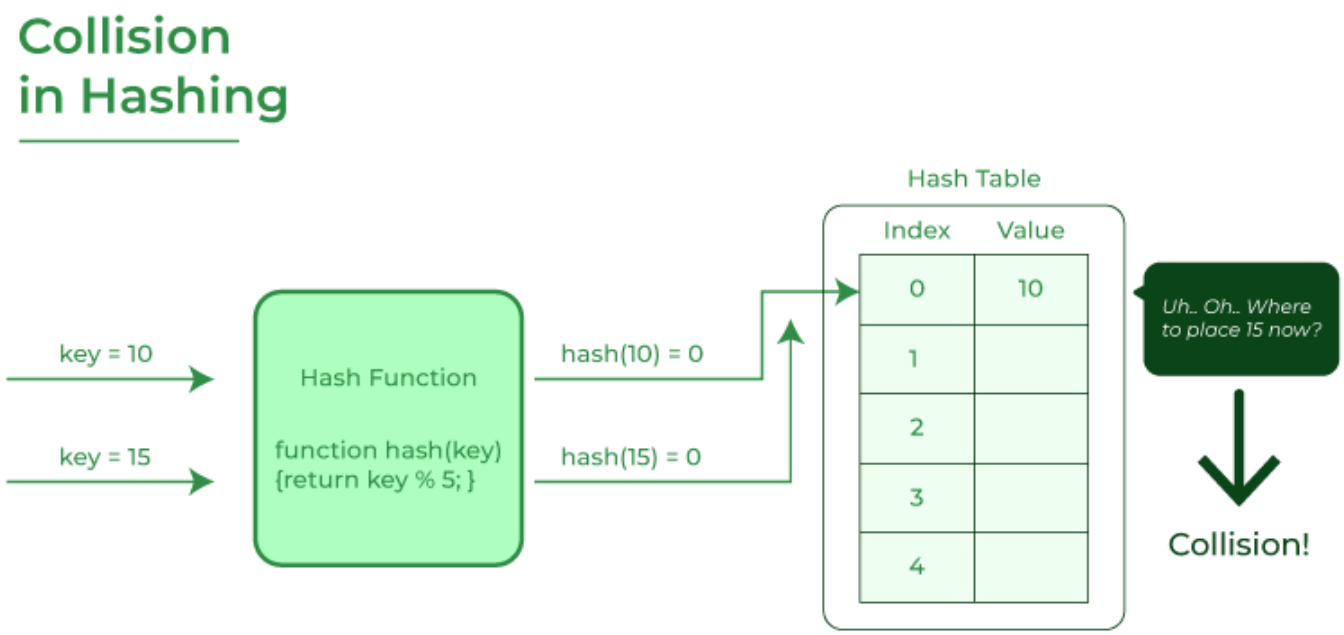
Bucket Overflow

If we want to insert some new record into the file but the address of a data bucket generated by the hash function is not empty, or data already exists in that address. This situation in the static hashing is known as bucket overflow. This is a critical situation in this method.

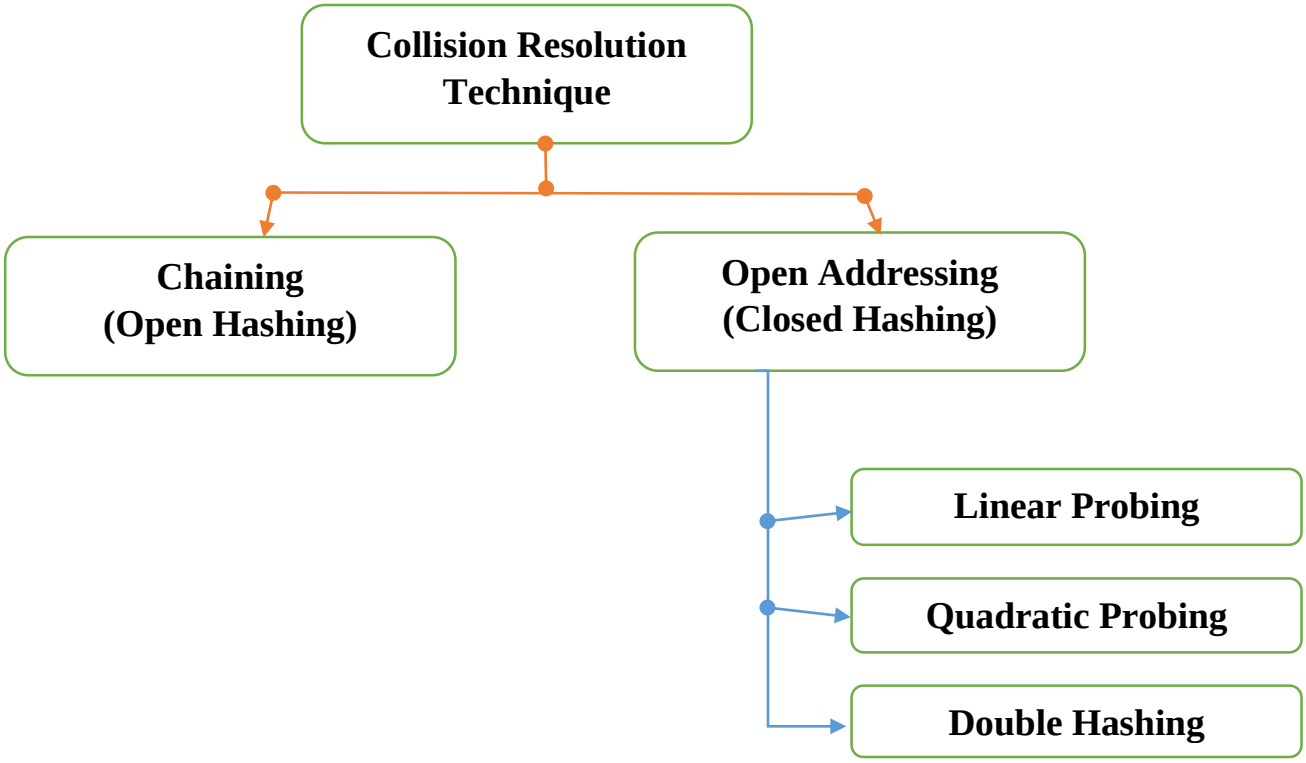
The condition of bucket-overflow is known as **collision**. This is a fatal state for any static hash function.

What is Collision?

The hashing process generates a small number for a big key, so there is a possibility that two keys could produce the same value. The situation where the newly inserted key maps to an already occupied, and it must be handled using some collision handling technology.



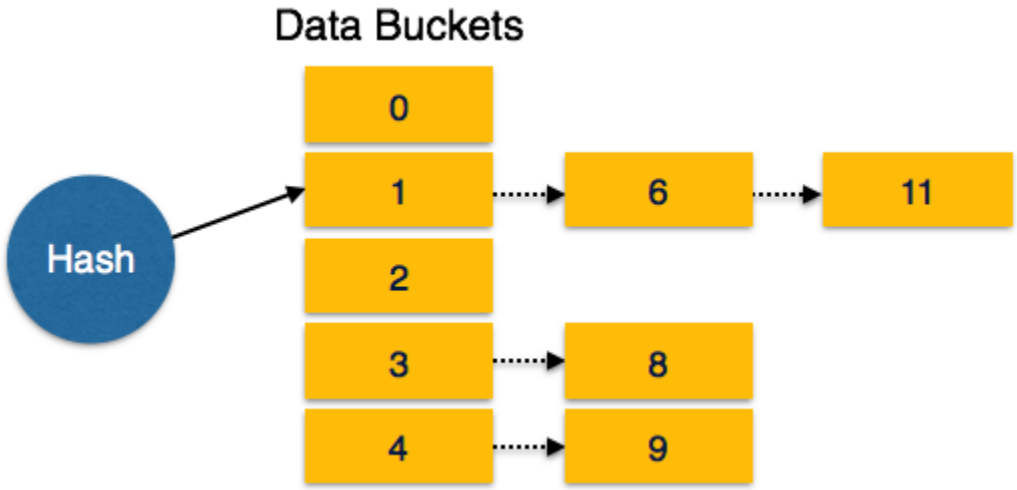
Collision Resolution Technique:



To overcome this situation, there are various methods. Some commonly used methods are as follows:

In this case, overflow chaining can be used.

- **Overflow Chaining** − When buckets are full, a new bucket is allocated for the same hash result and is linked after the previous one. This mechanism is called **Open Hashing**.



Open Addressing:

1. **Linear Probing** − When a hash function generates an address at which data is already stored, the next free bucket is allocated to it. This mechanism is called **Open Addressing**.

   In linear probing, the hash table is searched sequentially that starts from the original location of the hash. If in case the location that we get is already occupied, then we check for the next location.

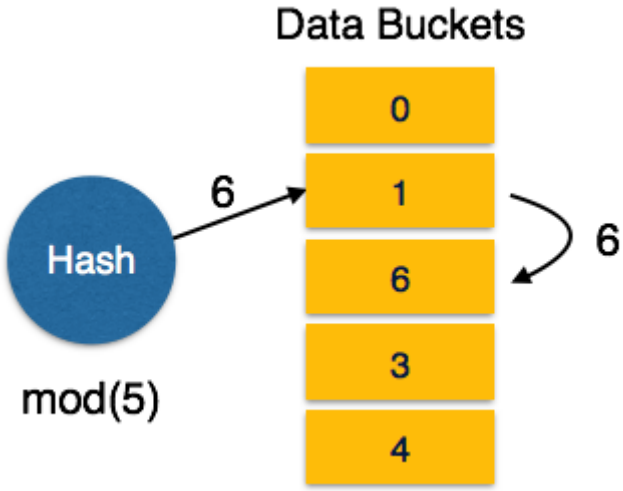


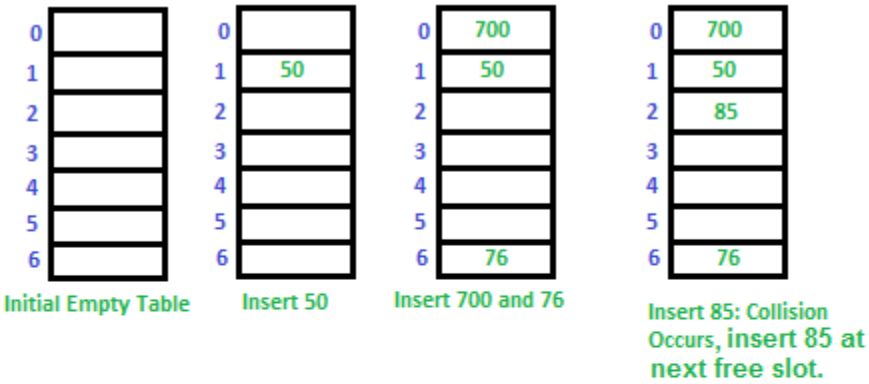
Fig: Linear Probing

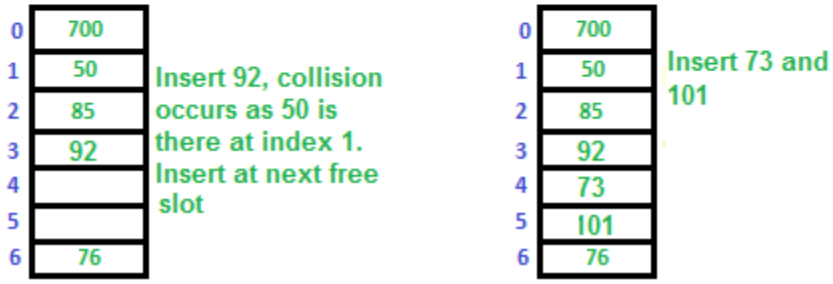
The function used for rehashing is as follows: rehash(key) = (n+1) %table-size.

For example, The typical gap between two probes is 1 as seen in the example below:
Let **hash(x)** be the slot index computed using a hash function and **S** be the table size
If slot hash(x) % S is full, then we try (hash(x) + 1) % S
If (hash(x) + 1) % S is also full, then we try (hash(x) + 2) % S
If (hash(x) + 2) % S is also full, then we try (hash(x) + 3) % S
……..
……..

Example: Let us consider a simple hash function as "key mod 7" and a sequence of keys as 50, 700, 76, 85, 92, 73, 101,

which means hash(key)= key% S, here S=size of the table =7, indexed from 0 to 6. We can define the hash function as per our choice if we want to create a hash table, although it is fixed internally with a pre-defined formula.





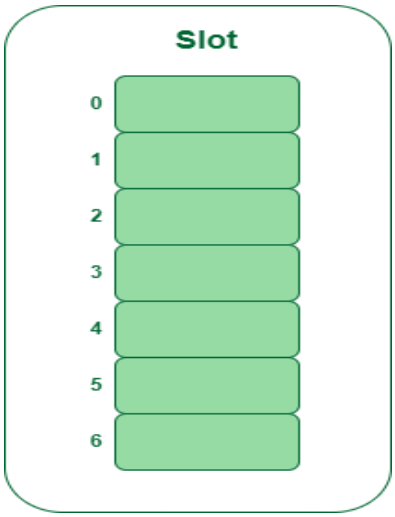
2. Quadratic Probing

If you observe carefully, then you will understand that the interval between probes will increase proportionally to the hash value. Quadratic probing is a method with the help of which we can solve the problem of clustering that was discussed above. This method is also known as the **mid-square** method. In this method, we look for the **i^2'th** slot in the **ith** iteration. We always start from the original hash location. If only the location is occupied then we check the other slots.
let hash(x) be the slot index computed using hash function.

If slot hash(x) % S is full, then we try (hash(x) + 1*1) % S
If (hash(x) + 1*1) % S is also full, then we try (hash(x) + 2*2) % S
If (hash(x) + 2*2) % S is also full, then we try (hash(x) + 3*3) % S
……..
……..

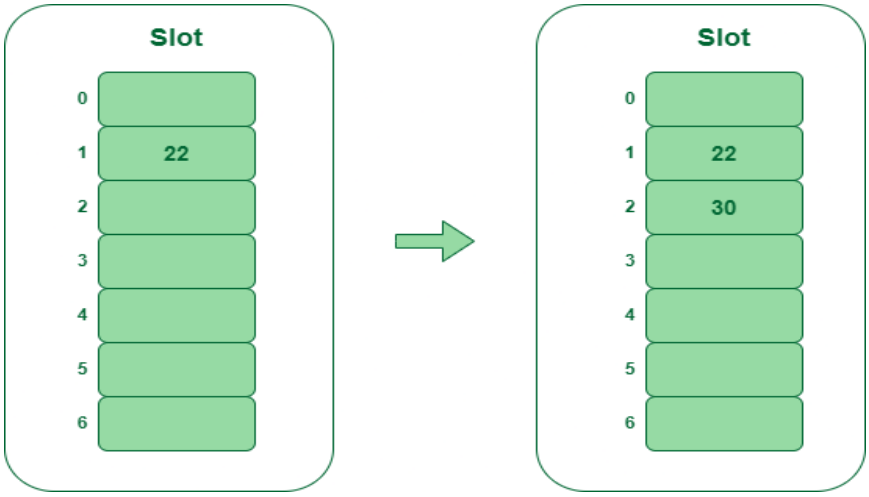
Example: Let us consider table Size = 7, hash function as Hash(x) = x % 7 and collision resolution strategy to be f(i) = i^2 . Insert = 22, 30, and 50.

- **Step 1:** Create a table of size 7.



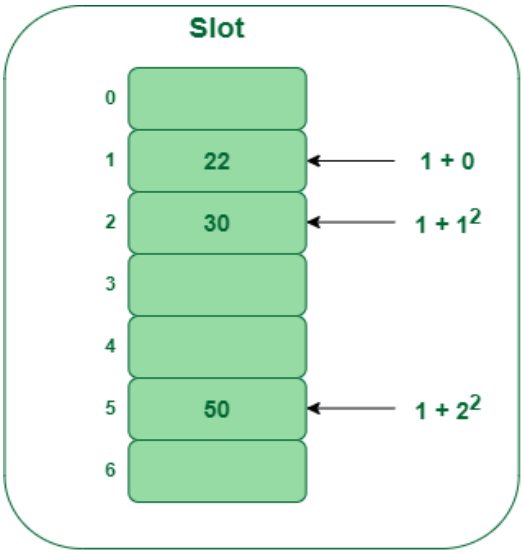
Hash table

- **Step 2** – Insert 22 and 30
    - Hash(22) = 22 % 7 = 1, Since the cell at index 1 is empty, we can easily insert 22 at slot 1.
    - Hash(30) = 30 % 7 = 2, Since the cell at index 2 is empty, we can easily insert 30 at slot 2.



Insert keys 22 and 30 in the hash table

- **Step 3:** Inserting 50
    - Hash(50) = 50 % 7 = 1
    - In our hash table slot 1 is already occupied. So, we will search for slot $1+1^2$, i.e. 1+1 = 2,
    - Again slot 2 is found occupied, so we will search for cell $1+2^2$, i.e.1+4 = 5,
    - Now, cell 5 is not occupied so we will place 50 in slot 5.



Insert key 50 in the hash table

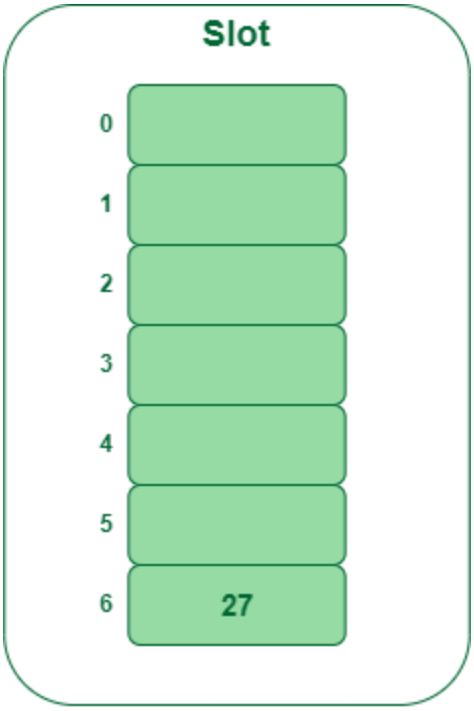
3. Double Hashing

The intervals that lie between probes are computed by another hash function. Double hashing is a technique that reduces clustering in an optimized way. In this technique, the increments for the probing sequence are computed by using another hash function. We use another hash function hash2(x) and look for the i*hash2(x) slot in the **ith** rotation.
let hash(x) be the slot index computed using hash function.

If slot hash(x) % S is full, then we try (hash(x) + 1*hash2(x)) % S
If (hash(x) + 1*hash2(x)) % S is also full, then we try (hash(x) + 2*hash2(x)) % S
If (hash(x) + 2*hash2(x)) % S is also full, then we try (hash(x) + 3*hash2(x)) % S
……..
……..

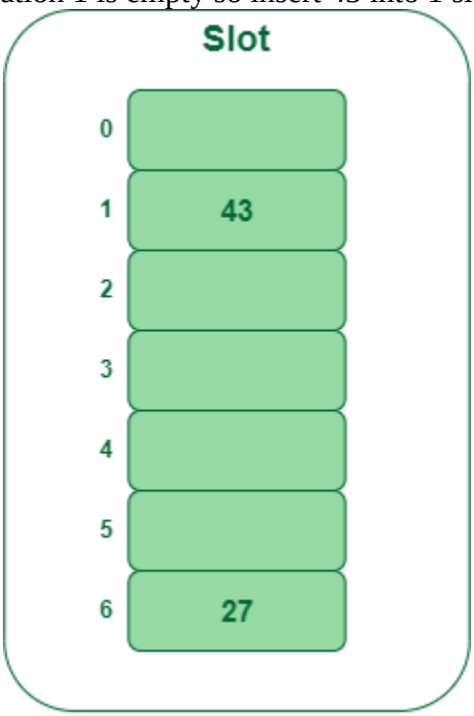
Example: Insert the keys 27, 43, 692, 72 into the Hash Table of size 7. where first hash-function is **h1(k) = k mod 7** and second hash-function is **h2(k) = 1 + (k mod 5)**

- **Step 1:** Insert 27
    - 27 % 7 = 6, location 6 is empty so insert 27 into 6 slot.



Insert key 27 in the hash table

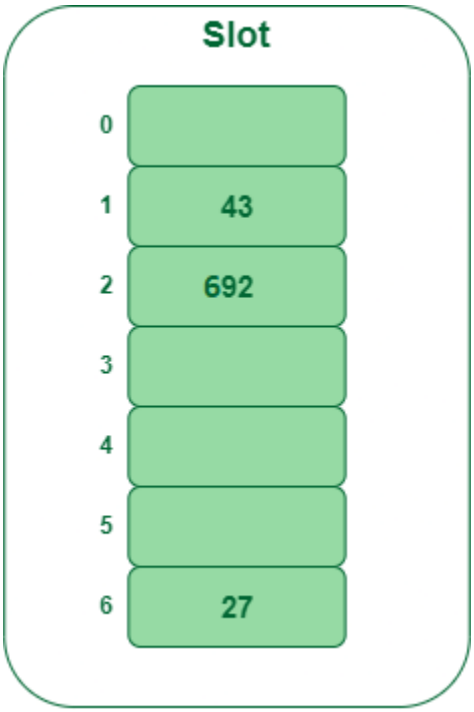
- **Step 2:** Insert 43
    - 43 % 7 = 1, location 1 is empty so insert 43 into 1 slot.



Insert key 43 in the hash table

- **Step 3:** Insert 692
    - 692 % 7 = 6, but location 6 is already being occupied and this is a collision
    - So, we need to resolve this collision using double hashing.

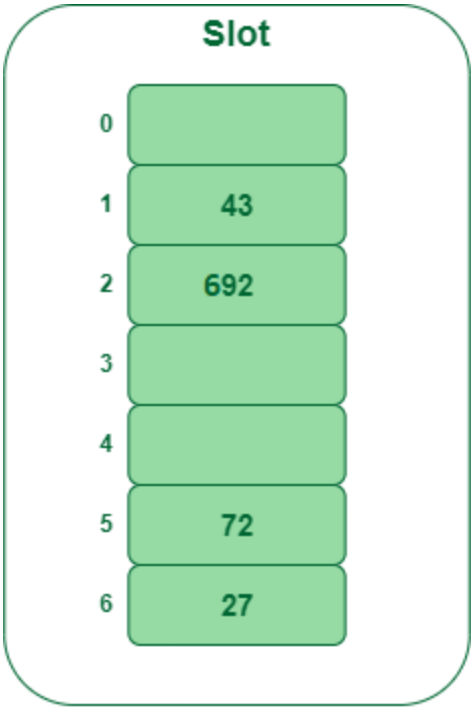
hnew = [h1(692) + i * (h2(692)] % 7
= [6 + 1 * (1 + 692 % 5)] % 7
= 9% 7
= 2
Now, as 2 is an empty slot,
so we can insert 692 into 2nd slot.



Insert key 692 in the hash table

- **Step 4:** Insert 72
    - 72 % 7 = 2, but location 2 is already being occupied and this is a collision.
    - So we need to resolve this collision using double hashing.

hnew = [h1(72) + i * (h2(72)] % 7
= [2 + 1 * (1 + 72 % 5)] % 7
= 5 % 7
= 5,
Now, as 5 is an empty slot,
so we can insert 72 into 5th slot.



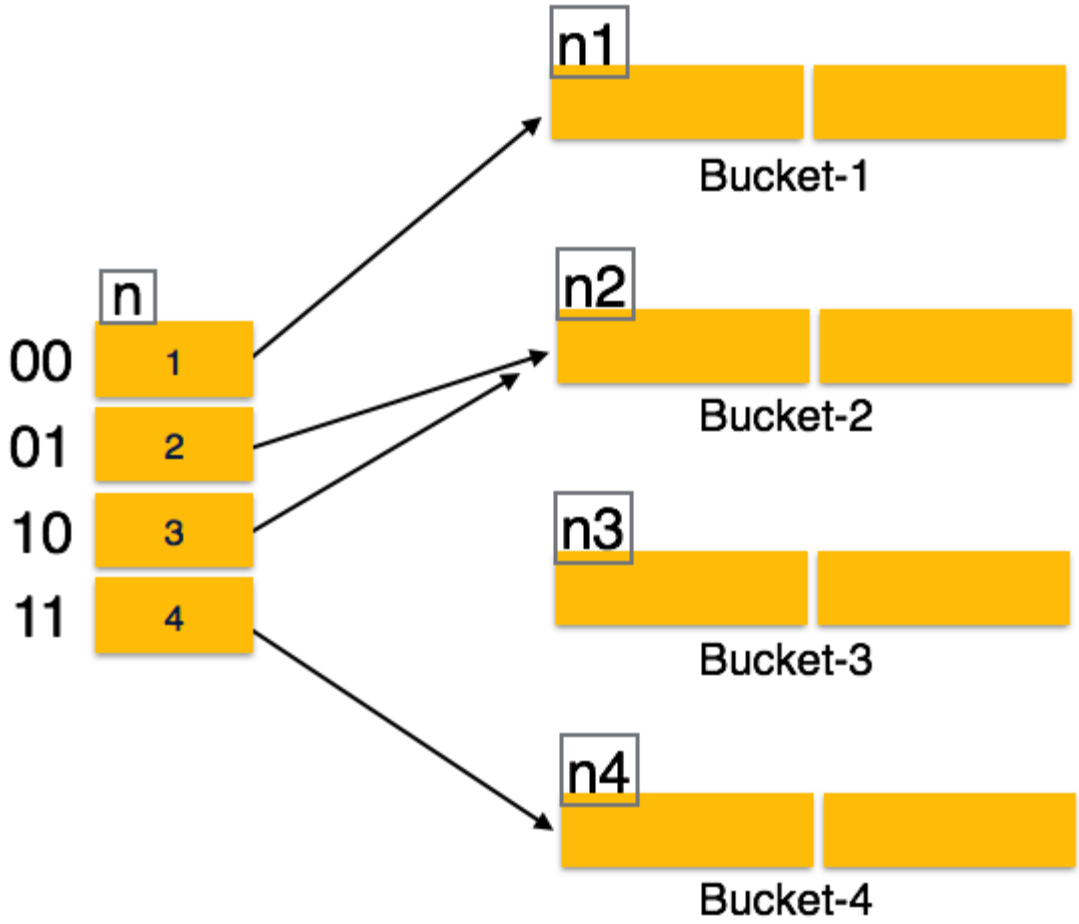
Insert key 72 in the hash table

See this for step-by-step diagrams:

Dynamic Hashing

The problem with static hashing is that it does not expand or shrink dynamically as the size of the database grows or shrinks. Dynamic hashing provides a mechanism in which data buckets are added and removed dynamically and on-demand. Dynamic hashing is also known as **extended hashing**.

Hash function, in dynamic hashing, is made to produce a large number of values and only a few are used initially.



Organization

The prefix of an entire hash value is taken as a hash index. Only a portion of the hash value is used for computing bucket addresses. Every hash index has a depth value to signify how many bits are used for computing a hash function. These bits can address 2n buckets. When all these bits are consumed − that is, when all the buckets are full − then the depth value is increased linearly and twice the buckets are allocated.

Operation

- **Querying** − Look at the depth value of the hash index and use those bits to compute the bucket address.
- **Update** − Perform a query as above and update the data.
- **Deletion** − Perform a query to locate the desired data and delete the same.
- **Insertion** − Compute the address of the bucket
    - If the bucket is already full.
        - Add more buckets.
        - Add additional bits to the hash value.
        - Re-compute the hash function.
    - Else
        - Add data to the bucket,
    - If all the buckets are full, perform the remedies of static hashing.

Hashing is not favorable when the data is organized in some ordering and the queries require a range of data. When data is discrete and random, hash performs the best.

Hashing algorithms have high complexity than indexing. All hash operations are done in constant time.

Problem 1: The keys 1, 3, 12, 4, 25, 6, 18, 20, 8 are inserted into empty hash table of length 10. Using open addressing with hash function $H(i) = i^2 \bmod 10$ and linear Probing. What is the resultant hash table and find the maximum probe value?

Solution:

Given,

Hash function $H(i) = i^2 \bmod 10$

Since, Hash function $H(i) = i^2 \bmod 10$. So, the hash table contain 0-9 indexing.

So, for key 1 : $H(1) = 1^2 \bmod 10 = 1$

Probe value: 1

- for key 3 : $H(3) = 3^2 \bmod 10$
  $= 9 \bmod 10 = 9$
  Probe value: 1
- for key 12 : $H(12) = 12^2 \bmod 10$
  $= 144 \bmod 10 = 4$
  Probe value: 1
- for key 4 : $H(4) = 4^2 \bmod 10$
  $= 16 \bmod 10 = 6$
  Probe value: 1
- for key 25 : $H(25) = 25^2 \bmod 10$
  $= 625 \bmod 10 = 5$
  Probe value: 1
- for key 6 : $H(6) = 6^2 \bmod 10$
  $= 36 \bmod 10 = 6$
  Probe value: 2

  Since, Bucket 6 already inserted a value. So, for linear probing key 6 will be inserted the next empty slot and it will be slot 7.
- for key 18 : $H(18) = 18^2 \bmod 10$
  $= 324 \bmod 10 = 4$
  Probe value: 5

  Since, Bucket 4 already inserted a value. So, for linear probing key 18 will be inserted the next empty slot and it will be slot 8.
- for key 20 : $H(20) = 20^2 \bmod 10$
  $= 400 \bmod 10 = 0$
  Probe value: 1
- for key 8 : $H(8) = 8^2 \bmod 10$
  $= 64 \bmod 10 = 4$
  Probe value: 9

  Since, Bucket 4 already inserted a value. So, for linear probing key 8 will be inserted the next empty slot and it will be slot 2.

20	0
1	1
8	2
	3
12	4
25	5
4	6
6	7
18	8
3	9

Hash Table

Maximum Probe Value: 9 for the key value 8